Data Structure : Assignment 1B

1. Explain the different types of data structures with examples.

Data structures are categorized into two main types: **Linear** and **Non-linear**.

- **Linear Data Structures**: Elements are arranged in a sequence.
    - **Array**: Fixed size, stores elements of the same type.
      Example: arr = [1, 2, 3, 4]
    - **List (Python)**: Dynamic array. Example: my_list = ["apple", "banana"]
    - **Stack**: Follows LIFO (Last In, First Out).
    - **Queue**: Follows FIFO (First In, First Out).
- **Non-linear Data Structures**: Elements are arranged hierarchically or in graphs.
    - **Tree**: Hierarchical structure (e.g., Binary Tree).
    - **Graph**: Set of nodes connected by edges.
    - **Heap**: A specialized tree-based structure satisfying the heap property.

---

2. What is recursion? How does it work? Explain with an example of a recursive function in Python.

Recursion is a technique where a function calls itself to solve smaller instances of the problem.

It works with two main components:

- **Base Case**: The stopping condition.
- **Recursive Case**: The function calls itself.

Example:

```python
def factorial(n):
    if n == 0:
        return 1  # base case
    return n * factorial(n - 1)  # recursive case

print(factorial(5))  # Output: 120
```

3. Explain the concept of time and space complexity with examples.

- **Time Complexity**: Measures the time taken as a function of input size n.
    - Example: O(n) for a linear search, where the algorithm checks each item once.
- **Space Complexity**: Measures the memory used relative to input size.
    - Example: A function storing all results in a list uses O(n) space.

---

4. Differentiate between Big O, Theta (Θ), and Omega (Ω) notations with suitable examples.

- **Big O (O)**: Worst-case scenario.
  Example: Linear search is O(n).
- **Theta (Θ)**: Average-case or tight bound.
  Example: Balanced binary search has Θ(log n).
- **Omega (Ω)**: Best-case scenario.
  Example: Linear search may find the target at first index → Ω(1).

---

5. Describe the importance of efficient memory utilization in data structures and how Python manages memory.

Efficient memory usage:

- Improves performance
- Reduces overhead
- Prevents memory leaks

Python Memory Management:

- Uses automatic garbage collection.
- Objects are managed by reference counting.

- Larger objects like lists are stored in heap memory.

---

6. What is a stack? Explain its working principle with an example in Python.

A **stack** is a LIFO (Last In, First Out) data structure.

Operations:

- push(): Add item
- pop(): Remove item
- peek(): View top item

Example:

```python
stack = []
stack.append(10)   # push
stack.append(20)
print(stack.pop()) # Output: 20
```

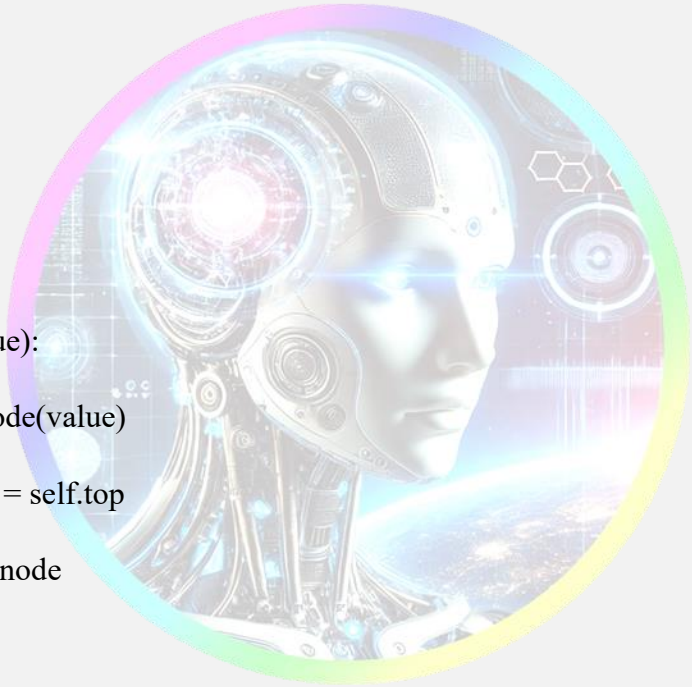
7. Explain how a stack can be implemented using both an array and a linked list in Python.

- **Using Array (List in Python)**:

```
stack = []
stack.append(1)
stack.pop()
```

Using Linked List:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    def push(self, value):
        new_node = Node(value)
        new_node.next = self.top
        self.top = new_node

    def pop(self):
        if self.top is None:
            return None
        value = self.top.data
        self.top = self.top.next
        return value
```



8. Describe real-life applications of stacks and how they are used in function calls and undo/redo operations.

- **Function Calls**: Uses call stack to manage local variables and return addresses.
- **Undo/Redo**: Undo pushes actions onto a stack, redo uses another stack to reverse undo.
- **Backtracking problems**: Like maze solving, recursion, etc.
- **Browser history**: Uses a stack to go back to the previous page.

---

9. Explain how to convert an infix expression to postfix using a stack with an example.

Steps:

1. Use a stack for operators.
2. Output operands immediately.
3. Apply precedence rules for operators.

Example: Infix: A + B * C

Postfix: A B C * +

Algorithm: Use the Shunting Yard algorithm by Dijkstra.

---

10. Describe the process of evaluating a postfix expression using a stack. Provide a Python implementation.

Steps:

1. Scan from left to right.
2. Push operands onto the stack.
3. On operator, pop two items, apply operation, push result.

Python Example:

```python
def evaluate_postfix(expr):
    stack = []
    for token in expr.split():
        if token.isdigit():
            stack.append(int(token))
        else:
            b = stack.pop()
            a = stack.pop()
            if token == '+':
                stack.append(a + b)
            elif token == '*':
                stack.append(a * b)
    return stack[0]

print(evaluate_postfix("2 3 4 * +"))  # Output: 14
```

11. What is a queue? Explain how it follows the FIFO (First-In, First-Out) principle.

A **queue** is a linear data structure that follows the **FIFO** principle, meaning the first element added is the first to be removed.

Operations:

- enqueue(): Add to rear
- dequeue(): Remove from front

Example in Python:

from collections import deque

queue = deque()

queue.append(1) # enqueue

queue.append(2)

print(queue.popleft()) # dequeue → 1

12. Explain the differences between simple queue, circular queue, and double-ended queue (deque).

- **Simple Queue**: Insertion at rear, deletion from front. FIFO structure.

- **Circular Queue**: Last position connects to the first to reuse space efficiently.
- **Deque (Double-Ended Queue)**: Insertion and deletion at both ends.

Type	Insert	Delete
Simple Queue	Rear	Front
Circular Queue	Rear (wraps)	Front (wraps)
Deque	Both ends	Both ends

13. Discuss the applications of queues in computer science and real life.

Applications:

- **CPU Scheduling**: Processes are queued by priority or time slice.
- **IO Buffers**: Queues manage data flow.
- **Printers**: Jobs are printed in the order they're queued.
- **Call centers**: Customers wait in queue for agents.
- **BFS Traversal**: Uses queue for visiting graph nodes level by level.

---

14. How is a priority queue different from a normal queue? Explain how a priority queue is implemented using a heap.

- **Normal Queue**: FIFO—removal is based on order of arrival.
- **Priority Queue**: Elements are removed based on priority, not arrival time.

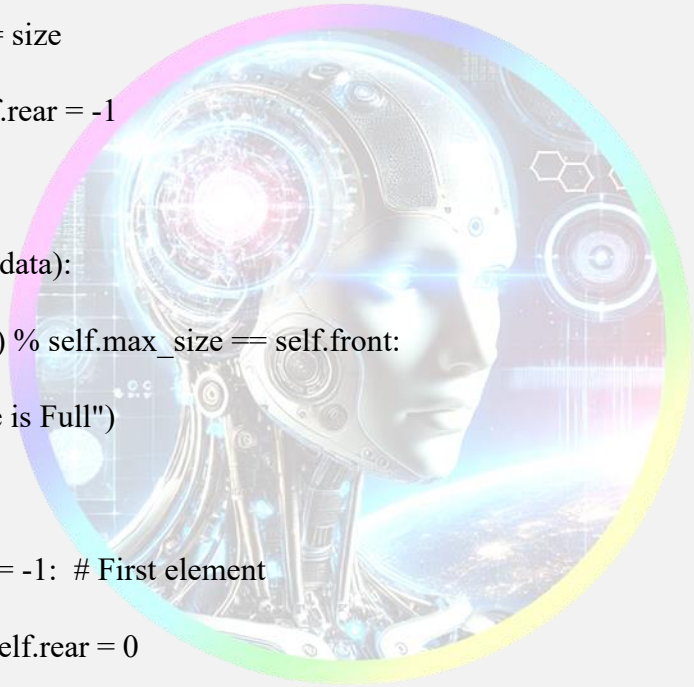
Heap Implementation (Min-Heap or Max-Heap):

```
import heapq

pq = []

heapq.heappush(pq, (1, "low"))
```

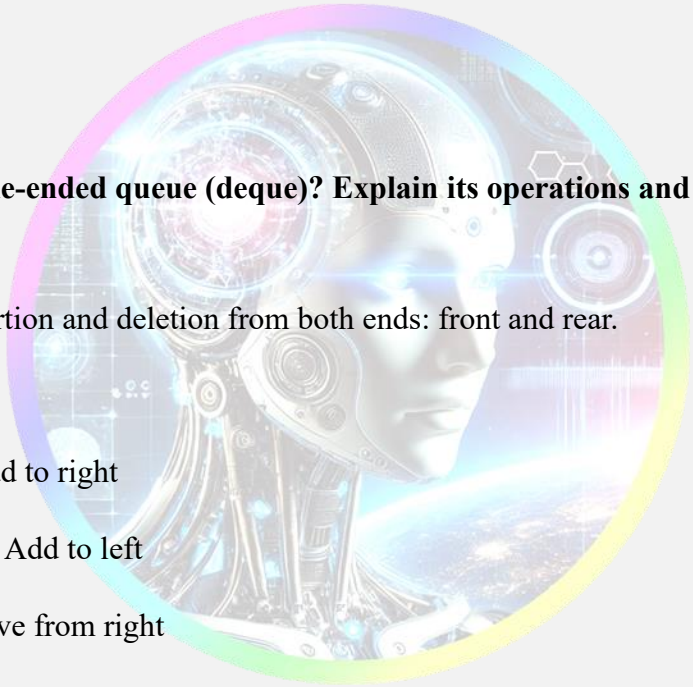
```
heapq.heappush(pq, (0, "high"))
print(heapq.heappop(pq))  # Output: (0, 'high')
```

15. Write a Python program to implement a circular queue using an array.

```
class CircularQueue:
    def __init__(self, size):
        self.queue = [None]*size
        self.max_size = size
        self.front = self.rear = -1

    def enqueue(self, data):
        if (self.rear + 1) % self.max_size == self.front:
            print("Queue is Full")
            return
        elif self.front == -1:  # First element
            self.front = self.rear = 0
        else:
            self.rear = (self.rear + 1) % self.max_size
        self.queue[self.rear] = data

    def dequeue(self):
        if self.front == -1:
            print("Queue is Empty")
```



```
        return
    elif self.front == self.rear:
        temp = self.queue[self.front]
        self.front = self.rear = -1
    else:
        temp = self.queue[self.front]
        self.front = (self.front + 1) % self.max_size
    return temp
```



16. What is a double-ended queue (deque)? Explain its operations and advantages over a normal queue.

A deque allows insertion and deletion from both ends: front and rear.

Operations:

- append(): Add to right
- appendleft(): Add to left
- pop(): Remove from right
- popleft(): Remove from left

Advantages:

- More flexible than queues.
- Useful for problems like sliding window, palindrome checking, etc.

---

17. Describe the implementation of multiple stacks and queues using a single array.

Concept:

- Divide the array logically.
- Use index manipulation to manage multiple stacks/queues.

Example (2 stacks in one array):

```
class TwoStacks:
    def __init__(self, size):
        self.arr = [None]*size
        self.top1 = -1
        self.top2 = size

    def push1(self, val):
        if self.top1 < self.top2 - 1:
            self.top1 += 1
            self.arr[self.top1] = val

    def push2(self, val):
        if self.top1 < self.top2 - 1:
            self.top2 -= 1
            self.arr[self.top2] = val
```

18. Explain how breadth-first search (BFS) algorithm uses queues for graph traversal.

BFS explores all neighbors of a node before going to the next level, and uses a queue to track the order of visiting.

Steps:

1. Enqueue starting node
2. Dequeue a node, mark as visited
3. Enqueue all unvisited neighbors

Example:

```python
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node)
            visited.add(node)
            queue.extend(graph[node])
```

19. Compare stacks and queues based on their operations, use cases, and performance.

Feature	Stack	Queue
Order	**LIFO**	**FIFO**
Operations	**push, pop**	**enqueue, dequeue**
Use Cases	**Undo, recursion**	**Scheduling, BFS**
Performance	**O(1) for main ops**	**O(1) for main ops**

Both are linear, but stack is better for backtracking, and queue is better for ordering tasks.

---

20. What are the advantages and disadvantages of using a linked list to implement a queue compared to using an array?

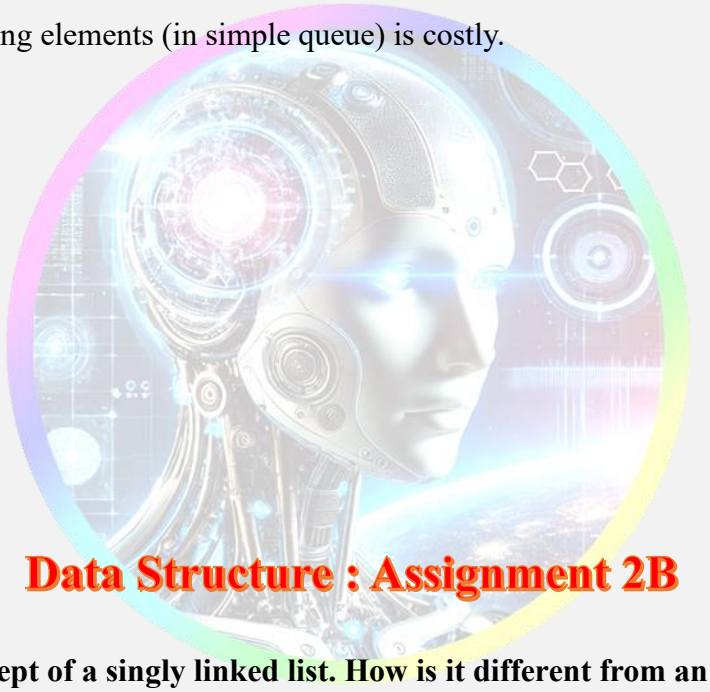
Linked List Queue:

- **Advantages:**
    - Dynamic size (no overflow unless memory full).
    - O(1) enqueue and dequeue.
- **Disadvantages:**

- Uses extra memory for pointers.
- Slightly slower due to pointer dereferencing.

Array Queue:

- **Advantages:**
  - Faster access time (direct index access).
- **Disadvantages:**
  - Fixed size unless using dynamic resizing.
  - Shifting elements (in simple queue) is costly.



Data Structure : Assignment 2B

1. Explain the concept of a singly linked list. How is it different from an array? Discuss its advantages and disadvantages.

A **singly linked list** is a linear data structure where each node contains **data** and a **reference to the next node**. It doesn't use contiguous memory.

Differences from Array:

- Linked list is **dynamic**; array has **fixed size**.
- Insertions/deletions are **faster** in linked lists (O(1) at head).
- Arrays allow **random access**; linked lists do not.

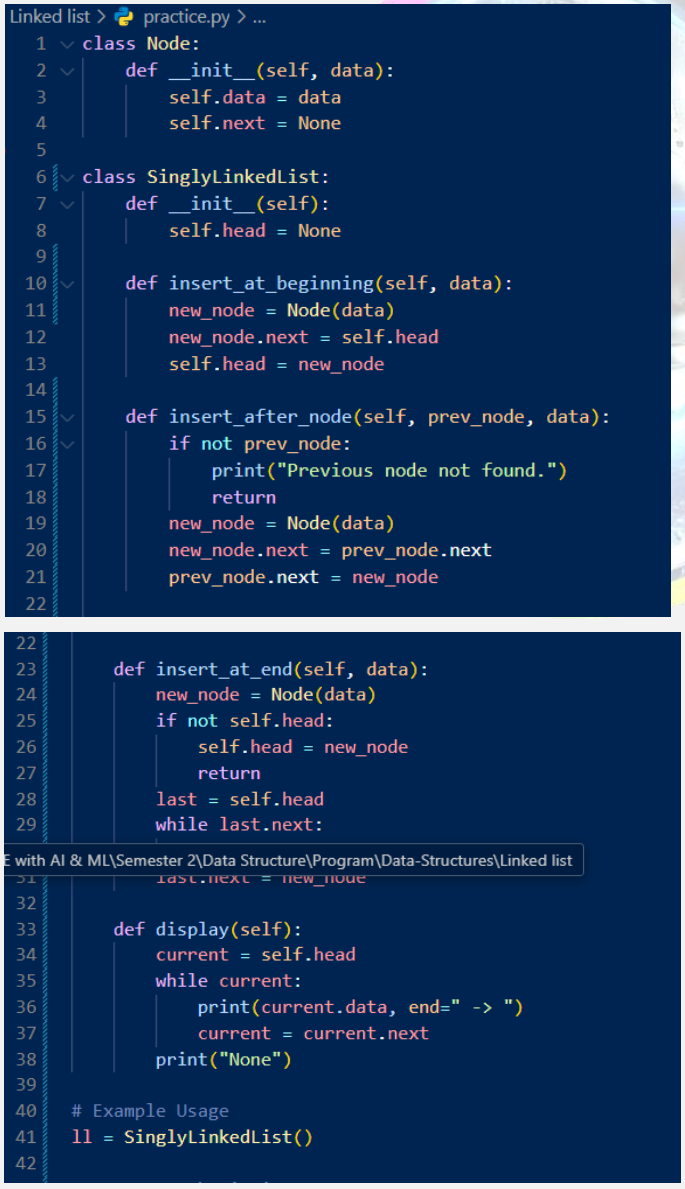
Advantages:

- Dynamic size
- Efficient insertions/deletions

Disadvantages:

- No direct access
- Extra memory for pointers

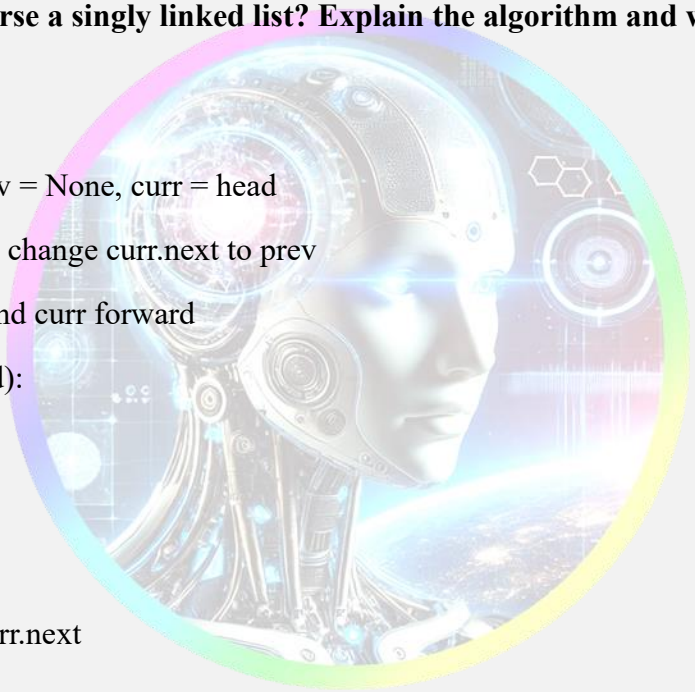
2. Describe the process of inserting a node at the beginning, middle, and end of a singly linked list. Provide Python code for each operation.



```
Linked list > practice.py > ...
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_after_node(self, prev_node, data):
        if not prev_node:
            print("Previous node not found.")
            return
        new_node = Node(data)
        new_node.next = prev_node.next
        prev_node.next = new_node

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last = self.head
        while last.next:
E with AI & ML\Semester 2\Data Structure\Program\Data-Structures\Linked list
            last = last.next
        last.next = new_node

    def display(self):
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")

# Example Usage
ll = SinglyLinkedList()
```

```
43  # Insert at beginning
44  ll.insert_at_beginning(10)
45  ll.insert_at_beginning(5)
46
47  # Insert at end
48  ll.insert_at_end(20)
49
50  # Insert after first node (after 5)
51  ll.insert_after_node(ll.head, 7)
52
53  # Display the linked list
54  ll.display()
55
```

3. How do you reverse a singly linked list? Explain the algorithm and write Python code to implement it.



Algorithm:

1. Initialize prev = None, curr = head
2. Traverse list: change curr.next to prev
3. Move prev and curr forward

```
def reverse_list(head):
    prev = None
    curr = head
    while curr:
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node
    return prev
```

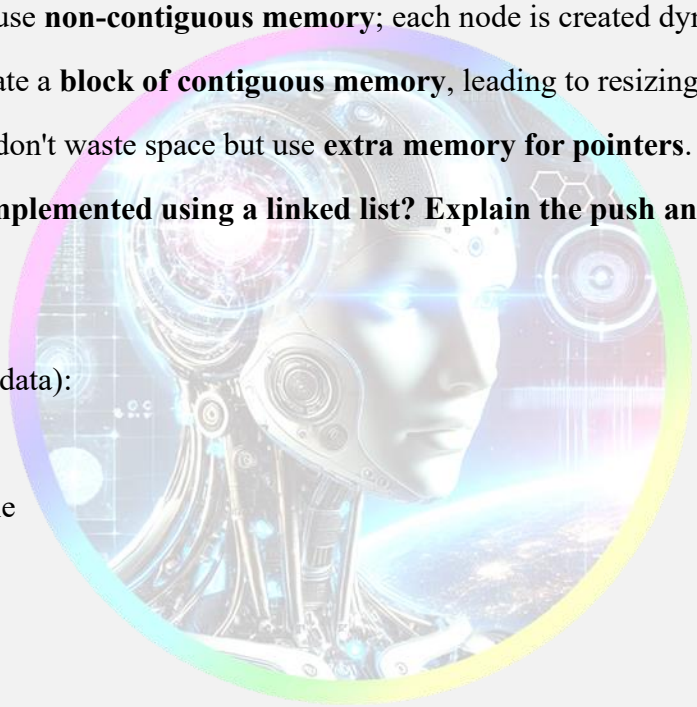
4. **What is the time complexity of various operations (insertion, deletion, searching, traversal) in a singly linked list? Justify your answer.**

Operation	Time Complexity	Reason
Insertion	O(1) at head, O(n) at end/middle	Traverse needed for non-head
Deletion	O(1) at head, O(n) at position	Need to find node
Searching	O(n)	Linear search
Traversal	O(n)	Visit every node

5. Explain how memory is allocated and managed in a singly linked list. How does it differ from contiguous memory allocation in arrays?

- Linked lists use **non-contiguous memory**; each node is created dynamically.
- Arrays allocate a **block of contiguous memory**, leading to resizing costs.
- Linked lists don't waste space but use **extra memory for pointers**.

6. How is a stack implemented using a linked list? Explain the push and pop operations with Python code.



```
class StackNode:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        node = StackNode(data)
        node.next = self.top
        self.top = node
```

```
    def pop(self):
        if not self.top:
            return None
        data = self.top.data
        self.top = self.top.next
        return data
```

7. What are the advantages of using a linked list to implement a stack over an array? Discuss with examples.

Advantages:

- Dynamic size: No overflow unless memory full
- No resizing cost like arrays

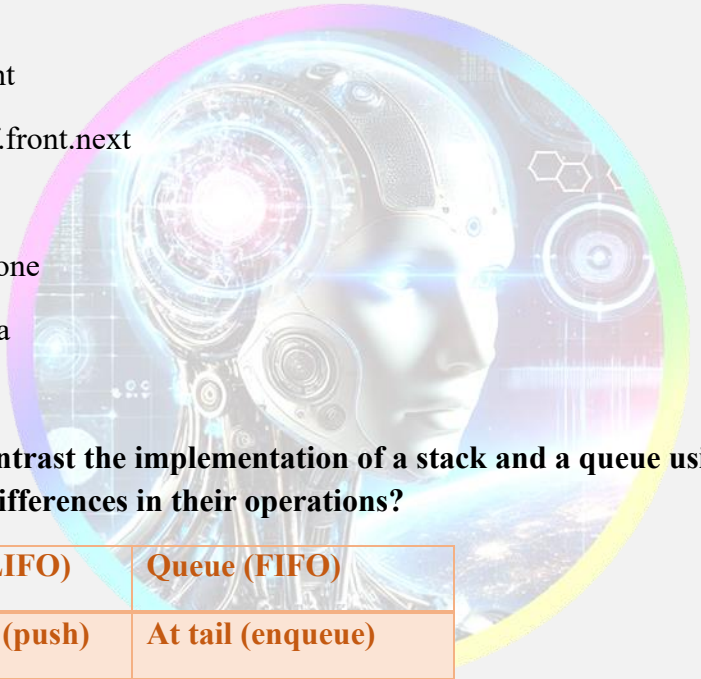
Example: If many unknown operations are performed, a linked list avoids reallocation issues present in arrays.

---

8. Explain how a queue is implemented using a linked list. Provide Python code for enqueue and dequeue operations.

```
class QueueNode:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = self.rear = None


    def enqueue(self, data):
        node = QueueNode(data)
```

```
        if not self.rear:
            self.front = self.rear = node
            return
        self.rear.next = node
        self.rear = node

    def dequeue(self):
        if not self.front:
            return None
        temp = self.front
        self.front = self.front.next
        if not self.front:
            self.rear = None
        return temp.data
```



9. Compare and contrast the implementation of a stack and a queue using a linked list. What are the key differences in their operations?

Feature	Stack (LIFO)	Queue (FIFO)
Insert	At head (push)	At tail (enqueue)
Delete	From head (pop)	From head (dequeue)
Use	Recursion, Undo	BFS, Scheduling

Main difference: Stack operates only at one end, Queue at both ends (insert rear, remove front).

10. What is the time complexity of stack and queue operations when implemented using a linked list? Justify your answer.

All basic operations in both stack and queue (insert, remove) are:

- Time Complexity: O(1)

- Justification: Insertion and deletion are done at head or tail using pointer updates, no traversal needed.

11. Explain how polynomials can be represented using linked lists. Provide an example and write Python code to create a polynomial.

Each node in the linked list stores a **coefficient** and an **exponent**. The list is usually sorted in descending order of exponents.

Example: $5x^3 + 2x^2 + 3$ can be represented as:

```python
class PolyNode:
    def __init__(self, coeff, expo):
        self.coeff = coeff
        self.expo = expo
        self.next = None

def create_polynomial(terms):
    head = None
    for coeff, expo in sorted(terms, key=lambda x: -x[1]):
        new_node = PolyNode(coeff, expo)
        new_node.next = head
        head = new_node
    return head
```

12. Describe the process of adding two polynomials using linked lists. Explain the algorithm and provide Python code for the same.

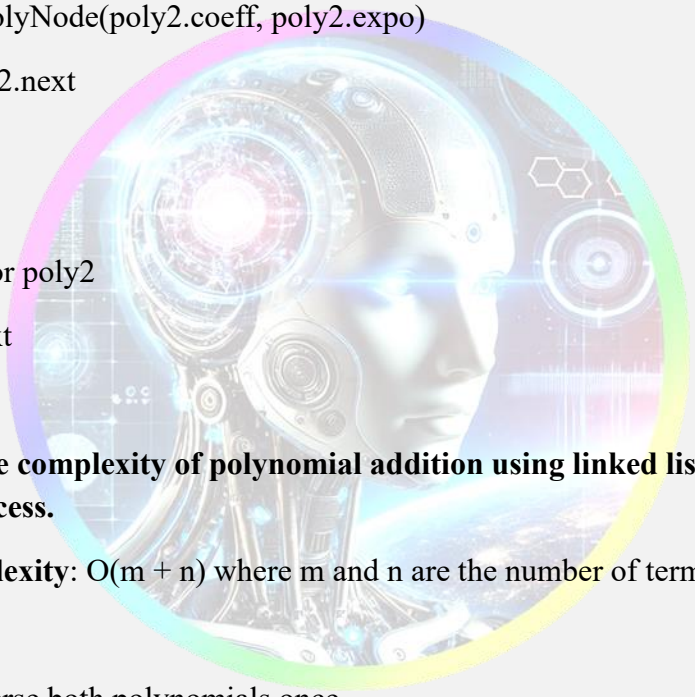
Algorithm:

- Traverse both lists simultaneously.
- If exponents match: add coefficients.
- Else: copy the node with the higher exponent.

Code:

```
def add_polynomials(poly1, poly2):
    dummy = PolyNode(0, 0)
    tail = dummy
```

```
while poly1 and poly2:
    if poly1.expo == poly2.expo:
        tail.next = PolyNode(poly1.coeff + poly2.coeff, poly1.expo)
        poly1, poly2 = poly1.next, poly2.next
    elif poly1.expo > poly2.expo:
        tail.next = PolyNode(poly1.coeff, poly1.expo)
        poly1 = poly1.next
    else:
        tail.next = PolyNode(poly2.coeff, poly2.expo)
        poly2 = poly2.next
    tail = tail.next

tail.next = poly1 or poly2
return dummy.next
```



13. What is the time complexity of polynomial addition using linked lists? Discuss the steps involved in the process.

- **Time Complexity**: O(m + n) where m and n are the number of terms in each polynomial.
- **Steps**:
    1. Traverse both polynomials once.
    2. Compare exponents.
    3. Merge nodes accordingly.

---

14. How do you handle exponents and coefficients while adding two polynomials using linked lists? Explain with an example.

- **Matching exponents**: add coefficients.
- **Different exponents**: keep the node with the higher exponent.

Example:

- $3x^2 + 2x + 1$ and $4x^2 + x \rightarrow 7x^2 + 3x + 1$

Handled in code by checking:

```
if p1.expo == p2.expo:
    result_coeff = p1.coeff + p2.coeff
```

15. What are the advantages of using linked lists for polynomial addition over arrays? Discuss with examples.

Advantages:

- No need for sparse storage; zero coefficients are skipped.
- Efficient insertion/deletion of terms.
- Handles large exponents without fixed-size arrays.

Example: Adding $x^{100} + 2x^5 + 3$ using an array requires size 101; linked list only needs 3 nodes.

---

16. What is a doubly linked list? How is it different from a singly linked list? Discuss its advantages and disadvantages.

A **doubly linked list (DLL)** has nodes with **three parts**: prev, data, and next.

Differences from SLL:

- DLL can be traversed both ways (forward and backward).
- Requires more memory (extra pointer).

Advantages:

- Easy backward traversal.
- Efficient deletion without needing previous node.

Disadvantages:

- More memory usage.
- Complex implementation.

---

17. Explain the process of inserting a node at the beginning, middle, and end of a doubly linked list. Provide Python code for each operation.

```python
class DNode:
    def __init__(self, data):
        self.data = data
        self.prev = self.next = None

# Insert at beginning
def insert_begin(head, data):
    node = DNode(data)
    node.next = head
    if head:
        head.prev = node
    return node

# Insert at end
def insert_end(head, data):
    node = DNode(data)
    if not head:
        return node
    temp = head
    while temp.next:
        temp = temp.next
    temp.next = node
    node.prev = temp
    return head
```

```python
# Insert after position
def insert_after(head, pos, data):
    temp = head
    for _ in range(pos):
        temp = temp.next
    node = DNode(data)
    node.next = temp.next
    node.prev = temp
    if temp.next:
        temp.next.prev = node
    temp.next = node
    return head
```

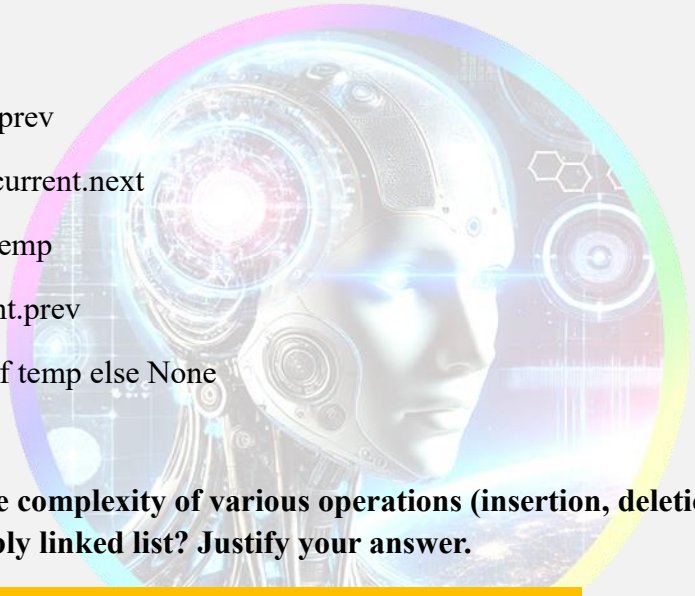
18. How do you reverse a doubly linked list? Explain the algorithm and write Python code to implement it.

Algorithm:

- Swap next and prev of each node.
- Update head to the last node.

Python Code:

```
def reverse_dll(head):
    temp = None
    current = head
    while current:
        temp = current.prev
        current.prev = current.next
        current.next = temp
        current = current.prev
    return temp.prev if temp else None
```



19. What is the time complexity of various operations (insertion, deletion, searching, traversal) in a doubly linked list? Justify your answer.

Operation	Time Complexity	Reason
Insertion	O(1) (if position known)	Adjust 2 pointers
Deletion	O(1) (if node given)	Use prev and next
Searching	O(n)	Linear traversal needed
Traversal	O(n)	Visit all nodes

20. Compare and contrast singly linked lists and doubly linked lists. When would you prefer one over the other?

Feature	Singly Linked List	Doubly Linked List
Memory Usage	Less (1 pointer)	More (2 pointers)
Traversal	One-direction	Bi-directional
Deletion	O(n) if no previous node	O(1) using prev pointer
Preferred when...	Memory is tight, simple use	Efficient deletion/traversal

Data Structure : Assignment 3B

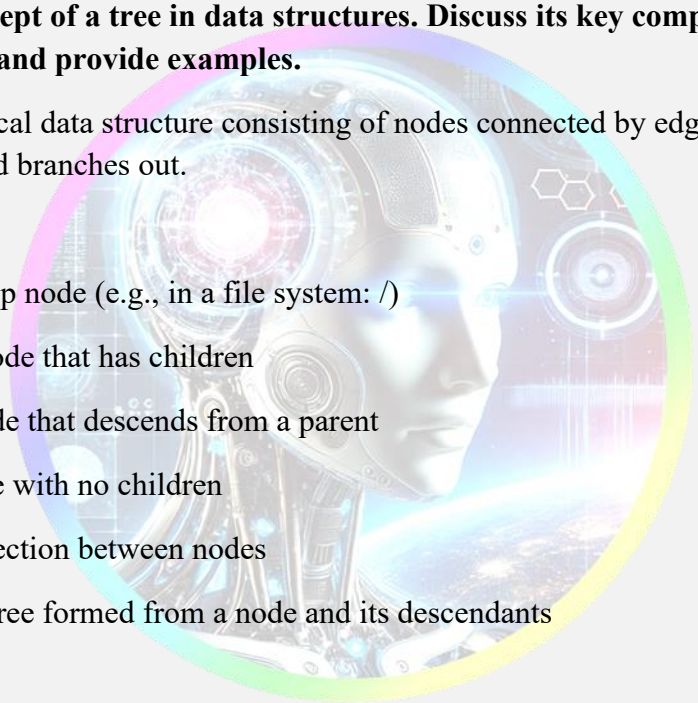
1. Explain the concept of a tree in data structures. Discuss its key components (root, leaf, parent, child, etc.) and provide examples.

A **tree** is a hierarchical data structure consisting of nodes connected by edges. It starts with a single **root node** and branches out.

Key components:

- **Root**: The top node (e.g., in a file system: /)
- **Parent**: A node that has children
- **Child**: A node that descends from a parent
- **Leaf**: A node with no children
- **Edge**: Connection between nodes
- **Subtree**: A tree formed from a node and its descendants

Example:



```
    A
   / \
  B   C
 / \
D   E
```

---

2. What is a binary tree? Explain its properties and how it differs from a general tree. Provide examples of full, complete, and perfect binary trees.

A **binary tree** is a tree where each node has at most **two children**: left and right.

Differences from general tree:

- General tree: any number of children
- Binary tree: max 2 children per node

Types:

- **Full**: Every node has 0 or 2 children
- **Complete**: All levels are full except possibly last, which is filled left to right
- **Perfect**: All internal nodes have 2 children and all leaves are at the same level

---

3. Describe the array and linked representations of a binary tree. What are the advantages and disadvantages of each representation?

Array Representation:

- Store nodes in level-order: root at index 0, left = 2i+1, right = 2i+2
- **Advantage**: Simple, fast indexing
- **Disadvantage**: Wastes space in sparse trees

Linked Representation:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

- **Advantage**: Memory-efficient for sparse trees
- **Disadvantage**: Slower access, complex to implement

---

4. Explain the concept of tree traversal. Compare and contrast inorder, preorder, postorder, and level-order traversals with examples.

Traversals visit every node in a specific order.

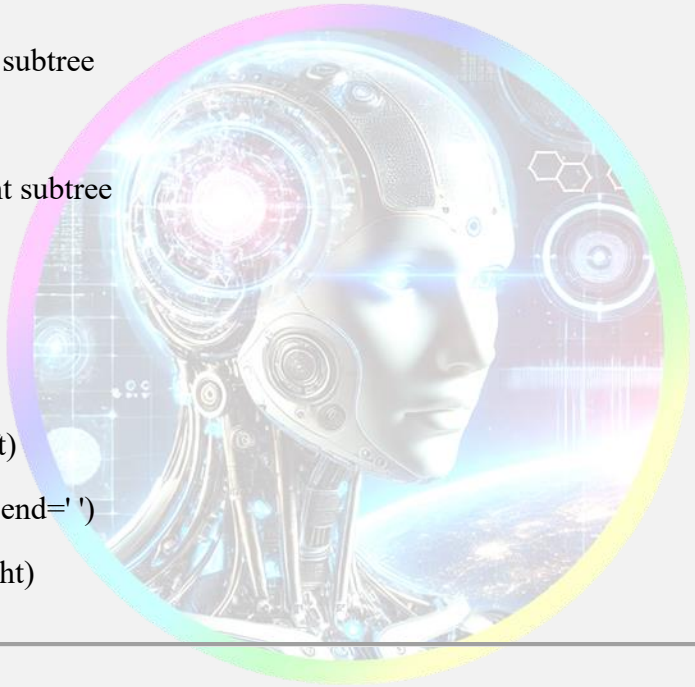
Type	Order	Example for tree A -> B, C
Inorder	Left, Root, Right	B A C
Preorder	Root, Left, Right	A B C
Postorder	Left, Right, Root	B C A
Level-order	Level by level (BFS)	A B C

---

5. Write a detailed algorithm for inorder traversal of a binary tree. Provide Python code to implement it.

Algorithm:

1. Traverse left subtree
2. Visit root
3. Traverse right subtree



Python:

```
def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=' ')
        inorder(root.right)
```

---

6. What is a binary search tree (BST)? Explain its properties and how it maintains order. Provide an example of inserting nodes into a BST.

A **BST** is a binary tree where:

- Left child < root < right child
- No duplicate values

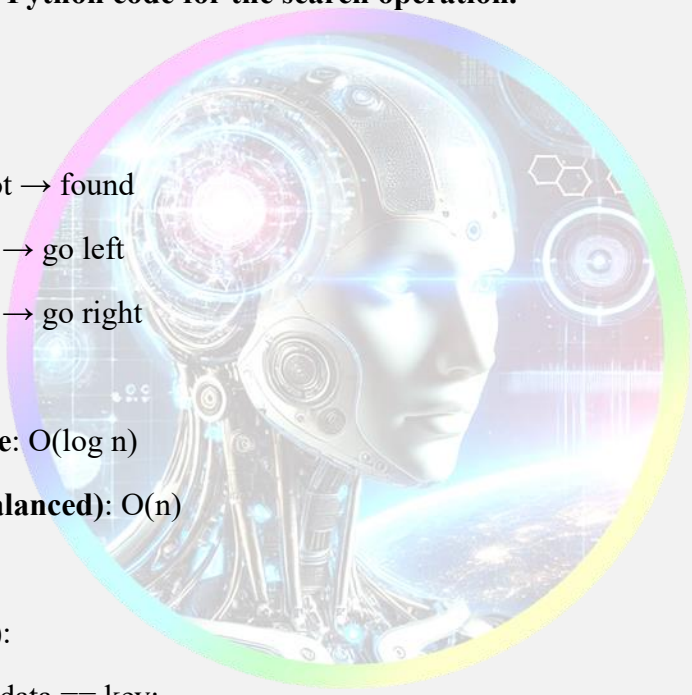
Insertion Example:
Insert 10, 5, 15, 3

```
def insert(root, key):
    if not root:
```

```
        return Node(key)
    if key < root.data:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root
```

---

7. Describe the process of searching for a node in a BST. What is the time complexity of this operation? Provide Python code for the search operation.



Search Algorithm:

- Start at root
- If key == root → found
- If key < root → go left
- If key > root → go right

Time Complexity:

- **Best/Average**: O(log n)
- **Worst (unbalanced)**: O(n)

Python:

```
def search(root, key):
    if not root or root.data == key:
        return root
    if key < root.data:
        return search(root.left, key)
    return search(root.right, key)
```

---

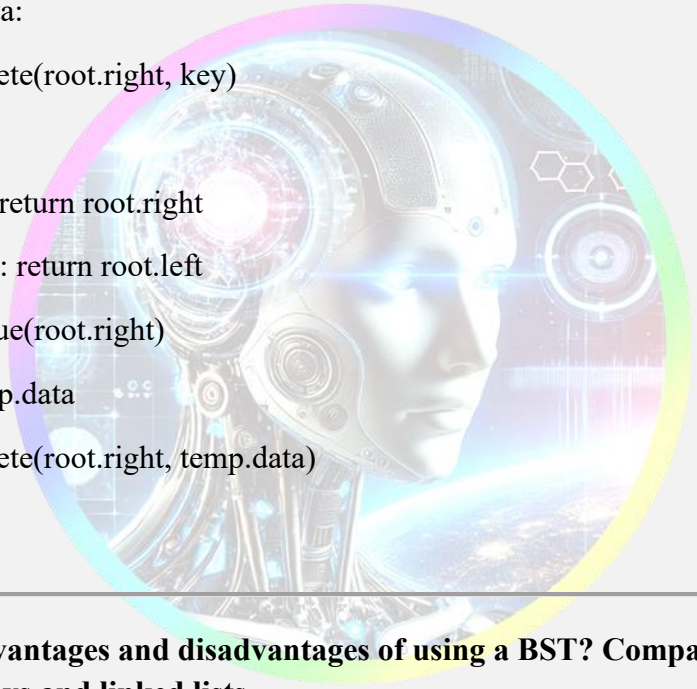
8. Explain the process of deleting a node from a BST. Discuss the three cases (node with no children, one child, and two children) with examples.

Cases:

1. **No child**: Simply delete the node
2. **One child**: Replace node with its child
3. **Two children**: Replace with inorder successor (or predecessor), then delete successor

Python:

```
def delete(root, key):
    if not root: return root
    if key < root.data:
        root.left = delete(root.left, key)
    elif key > root.data:
        root.right = delete(root.right, key)
    else:
        if not root.left: return root.right
        if not root.right: return root.left
        temp = minValue(root.right)
        root.data = temp.data
        root.right = delete(root.right, temp.data)
    return root
```



---

9. What are the advantages and disadvantages of using a BST? Compare it with other data structures like arrays and linked lists.

Advantages:

- Dynamic size
- Efficient search, insert, delete (O(log n))

Disadvantages:

- Unbalanced BST degrades to linked list (O(n))

Compared to:

- **Arrays**: Fast access, slow insert/delete

- **Linked lists**: Slow search, easy insert/delete
- **BST**: Balanced in features, but needs balance (e.g., AVL)

---

10. Write a detailed algorithm to check if a binary tree is a valid BST. Provide Python code to implement it.

Algorithm: Check that all left values < node and right values > node using recursion with bounds.

Python:

```
def isBST(root, min_val=float('-inf'), max_val=float('inf')):
    if not root:
        return True
    if not (min_val < root.data < max_val):
        return False
    return isBST(root.left, min_val, root.data) and isBST(root.right, root.data, max_val)
```

11. What is an AVL tree? Explain the concept of balance factor and how AVL trees maintain balance. Provide an example of AVL tree rotations.

An **AVL tree** is a self-balancing binary search tree (BST) where the **balance factor** (difference in height between left and right subtree) is always **-1, 0, or 1**.

Balance Factor = height(left subtree) - height(right subtree)

When the balance factor goes out of bounds after insertion or deletion, AVL performs **rotations** to restore balance.

Example: Inserting nodes 10, 20, 30 causes imbalance and triggers a **Left Rotation** on 10.

---

12. Describe the four types of rotations (LL, RR, LR, RL) used in AVL trees. When is each rotation applied? Provide examples.

1. **LL (Left-Left)**: Insertion in left subtree of left child → **Right rotation**
2. **RR (Right-Right)**: Insertion in right subtree of right child → **Left rotation**

3. **LR (Left-Right)**: Insertion in right subtree of left child → **Left rotation** on child, then **Right**
4. **RL (Right-Left)**: Insertion in left subtree of right child → **Right rotation** on child, then **Left**

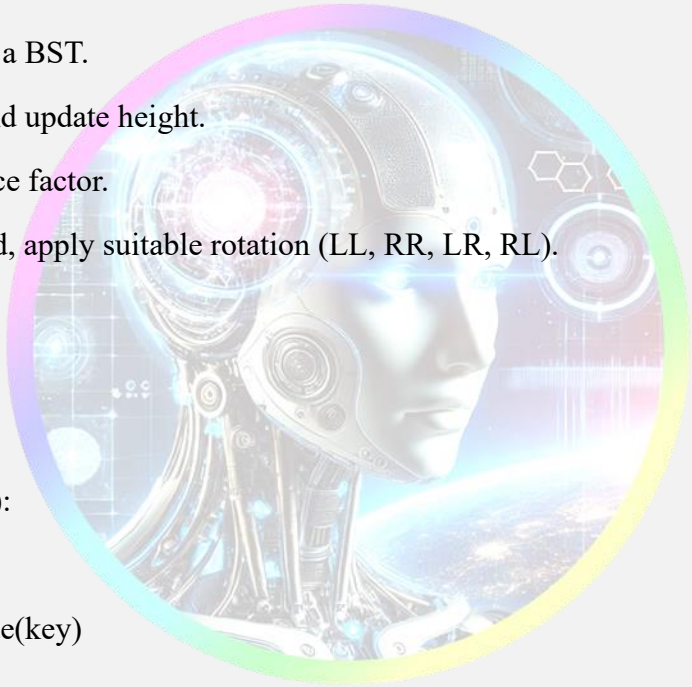
Example (LL):
Inserting 30, 20, 10 → imbalance at 30 (left-heavy) → right rotate at 30.

---

13. Explain the process of inserting a node into an AVL tree. How does the tree maintain its balance after insertion?

Steps:

1. Insert like in a BST.
2. Backtrack and update height.
3. Check balance factor.
4. If unbalanced, apply suitable rotation (LL, RR, LR, RL).



Python Snippet:

python

CopyEdit

```
def insert(node, key):
    if not node:
        return AVLNode(key)
    if key < node.key:
        node.left = insert(node.left, key)
    else:
        node.right = insert(node.right, key)
    update_height(node)
    return rebalance(node)
```

---

14. What is the time complexity of operations (insertion, deletion, searching) in an AVL tree? Justify your answer.

- **Insertion, Deletion, Search**: O(log n)
- AVL trees stay balanced, keeping height = O(log n)
- Rotations are constant time

Why balanced?
Rebalancing ensures every operation doesn't degenerate into a linked list.

---

15. Compare and contrast AVL trees with binary search trees. When would you prefer one over the other?

Feature	BST	AVL Tree
Balance	Not guaranteed	Always balanced
Time complexity	O(n) worst-case	O(log n) always
Rotations	No	Yes (to maintain balance)
Use case	Fast insert-heavy tasks	Search-heavy, balanced lookups

Prefer AVL when search time is critical.
Prefer BST if simplicity or frequent insertions matter.

---

16. What is a B-tree? Explain its properties and how it differs from a binary search tree. Provide an example of a B-tree.

A **B-tree** is a self-balancing **multi-way search tree** used in databases and file systems.

Properties:

- Each node can have multiple keys and children.
- Nodes are kept sorted.
- All leaves are at the same level.
- No node exceeds 2t - 1 keys (where t is minimum degree)

Difference from BST:

- BST: 2 children max
- B-tree: many children per node → better disk usage

Example (t = 2):
A node can have 1 to 3 keys and 2 to 4 children.

---

17. Describe the process of inserting a node into a B-tree. How does the tree maintain its balance after insertion?

Steps:

1. Insert key into correct node (like binary search).
2. If node overflows (more than 2t - 1 keys), **split**:
    - Middle key moves up.
    - Left and right halves become new children.

Maintains balance by promoting middle key and keeping tree height small.

---

18. What is the primary use of B-trees? Explain how they are used in database indexing and file systems.

Primary Use: Efficient **disk-based** storage operations.

Applications:

- **Database indexes**: Store and lookup keys efficiently.
- **File systems**: Manage directories and files (e.g., NTFS, HFS+)

Why?

- Minimize disk I/O by reducing tree height.
- High branching factor → fewer reads.

---

19. Compare and contrast B-trees with AVL trees. What are the advantages of B-trees for large datasets?

Feature	AVL Tree	B-tree
Node type	Binary (2 children)	Multiway (many children)
Memory use	In-memory	Disk-based

Feature	AVL Tree	B-tree
Speed	Fast in RAM	Optimized for disk access
Preferred for	RAM structures	Large datasets on disk

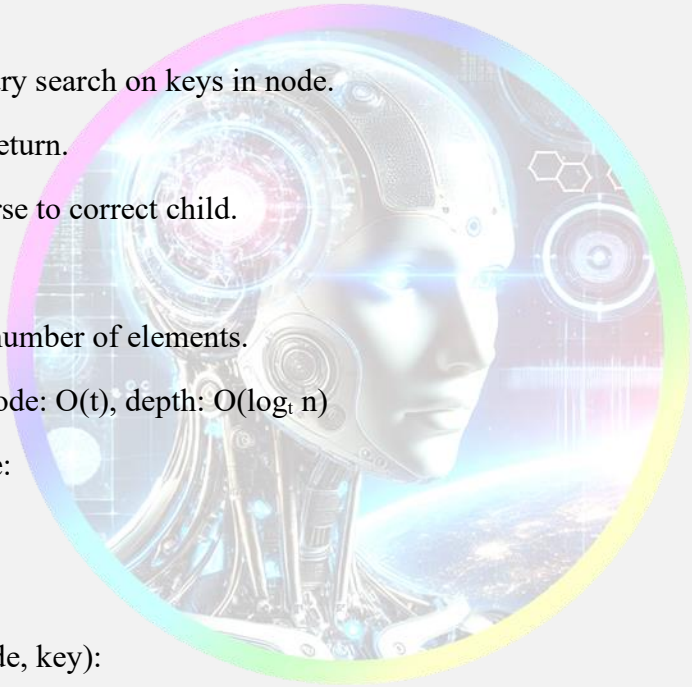
B-tree Advantage: Lower height → fewer disk reads → better for huge datasets.

---

20. Write a detailed algorithm for searching a key in a B-tree. What is the time complexity of this operation?

Algorithm:

1. Start at root.
2. Perform binary search on keys in node.
3. If found → return.
4. Else → recurse to correct child.



Time Complexity:

- **O(log n)** in number of elements.
- Search per node: O(t), depth: $O(\log_t n)$

Python Pseudocode:

python

CopyEdit

```
def search_btree(node, key):
    i = 0
    while i < len(node.keys) and key > node.keys[i]:
        i += 1
    if i < len(node.keys) and key == node.keys[i]:
        return node
    if node.leaf:
        return None
    return search_btree(node.children[i], key)
```

Data Structure : Assignment 4B

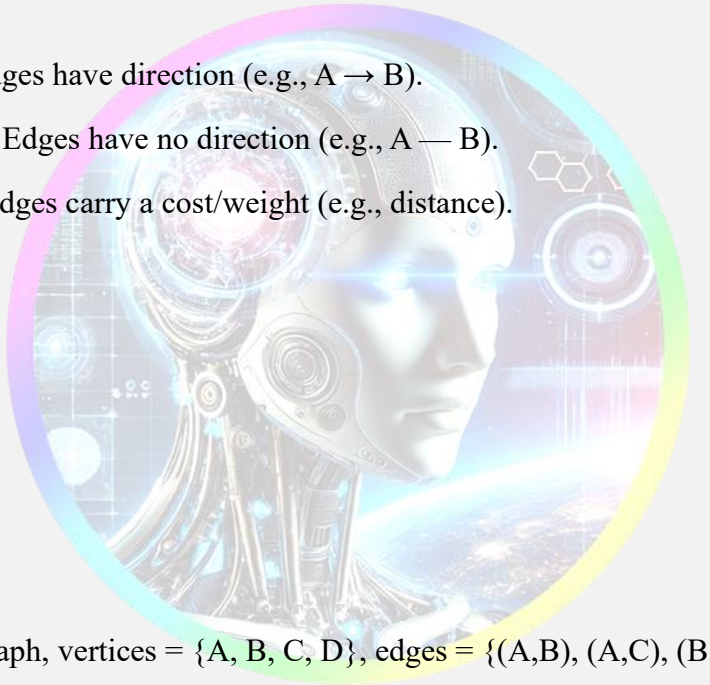
1. Explain the concept of a graph. Discuss its key components (vertices, edges, etc.) and provide examples.

A **graph** is a data structure consisting of **vertices (nodes)** and **edges (connections)**. It is used to represent relationships or networks.

- **Vertices (V)**: Entities or points in the graph.
- **Edges (E)**: Links between vertices.

Types of graphs:

- **Directed**: Edges have direction (e.g., A → B).
- **Undirected**: Edges have no direction (e.g., A — B).
- **Weighted**: Edges carry a cost/weight (e.g., distance).



Example:

plaintext

CopyEdit

A --- B

| |

C --- D

In this undirected graph, vertices = {A, B, C, D}, edges = {(A,B), (A,C), (B,D), (C,D)}.

---

2. Compare and contrast adjacency matrix and adjacency list representations of a graph. What are the advantages and disadvantages of each?

Feature	Adjacency Matrix	Adjacency List
Structure	2D array (V×V)	List of neighbors
Space	$O(V^2)$	O(V + E)
Lookup	O(1)	O(V)
Edge iteration	O(V)	O(#neighbors)

Best for	Dense graphs	Sparse graphs

Example:
Adjacency Matrix for 3 vertices:

0 1 2

0 0 1 0

1 1 0 1

2 0 1 0

Adjacency List:

{

0: [1],

1: [0, 2],

2: [1]

}

---

3. Explain BFS and DFS traversal algorithms. Compare their time and space complexities.

BFS (Breadth-First Search):

- Explores neighbors level by level.
- Uses a **queue**.

DFS (Depth-First Search):

- Explores as deep as possible before backtracking.
- Uses a **stack** (recursion or explicit).

	BFS	DFS
Time	O(V + E)	O(V + E)
Space	O(V)	O(V)
Use Case	Shortest path in unweighted graphs	Path existence, cycle detection

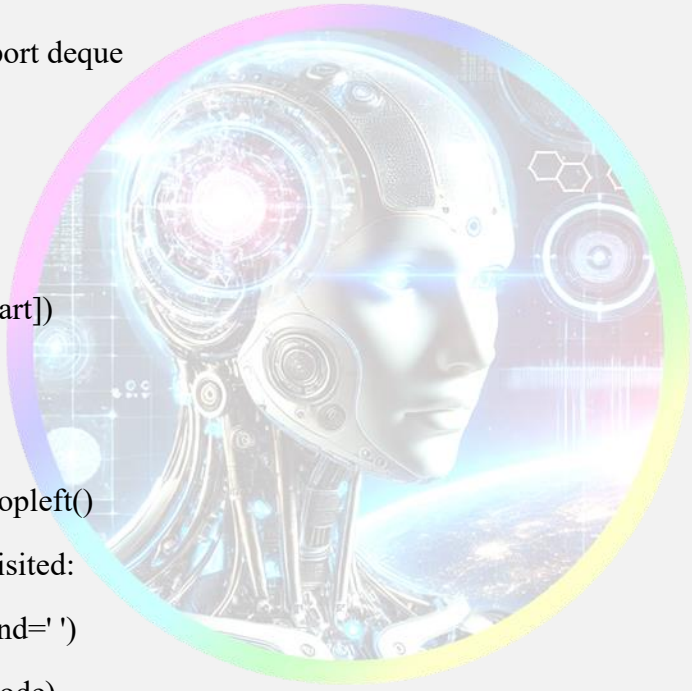
---

4. Write a detailed algorithm for BFS traversal of a graph. Provide Python code to implement it.

Algorithm:

1. Start at source node.
2. Mark it visited and enqueue.
3. While queue is not empty:
    - Dequeue a node.
    - Visit all unvisited neighbors and enqueue them.

Python Code:



```
from collections import deque


def bfs(graph, start):
    visited = set()
    queue = deque([start])


    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node, end=' ')
            visited.add(node)
            queue.extend(graph[node])
```

---

5. Write a detailed algorithm for DFS traversal of a graph. Provide Python code to implement it.

Algorithm:

1. Start at source node.
2. Mark it visited.
3. Recursively visit all unvisited neighbors.

Python Code (recursive):

```
def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()
    if node not in visited:
        print(node, end=' ')
        visited.add(node)
        for neighbor in graph[node]:
            dfs(graph, neighbor, visited)
```

---

6. Explain Dijkstra's algorithm for finding the shortest path in a graph. Provide an example and Python code.

Dijkstra's Algorithm finds the shortest path from a source to all nodes in a **weighted, non-negative graph**.

Steps:

1. Initialize distances (source=0, others=∞).
2. Use a min-heap to pick the node with the smallest distance.
3. Update distances of its neighbors.
4. Repeat until all nodes are processed.

Python Code (simplified):

```
import heapq


def dijkstra(graph, start):
    dist = {node: float('inf') for node in graph}
    dist[start] = 0
    heap = [(0, start)]


    while heap:
```

```
        d, node = heapq.heappop(heap)
        for neighbor, weight in graph[node]:
            new_dist = d + weight
            if new_dist < dist[neighbor]:
                dist[neighbor] = new_dist
                heapq.heappush(heap, (new_dist, neighbor))
    return dist
```

---

7. What is the time complexity of Dijkstra's algorithm? Justify your answer.

- **With binary heap**: O((V + E) log V)
    - Each node is pushed/popped once → O(V log V)
    - Each edge is relaxed → O(E log V)
- **Justification**: Efficient for sparse graphs due to log operations on heap.

---

8. Explain Prim's algorithm for finding the minimum spanning tree. Provide an example and Python code.

Prim's Algorithm builds a **Minimum Spanning Tree (MST)** by starting with one node and adding the smallest edge that connects to the tree.

Steps:

1. Start with any node.
2. Use a priority queue to select the smallest edge.
3. Add its connected node to MST.
4. Repeat until all nodes are included.

Python Code:

```
import heapq


def prim(graph, start):
    visited = set([start])
```

```
    edges = [(weight, start, v) for v, weight in graph[start]]
    heapq.heapify(edges)
    mst = []

    while edges:
        weight, u, v = heapq.heappop(edges)
        if v not in visited:
            visited.add(v)
            mst.append((u, v, weight))
            for neighbor, wt in graph[v]:
                if neighbor not in visited:
                    heapq.heappush(edges, (wt, v, neighbor))
    return mst
```

---

9. What is the time complexity of Prim's algorithm? Justify your answer.

- **With Min-Heap**: O(E log V)
    - E edges processed using heap
    - For each node, heap operations take O(log V)

---

10. Explain Kruskal's algorithm for finding the minimum spanning tree. Provide an example and Python code.

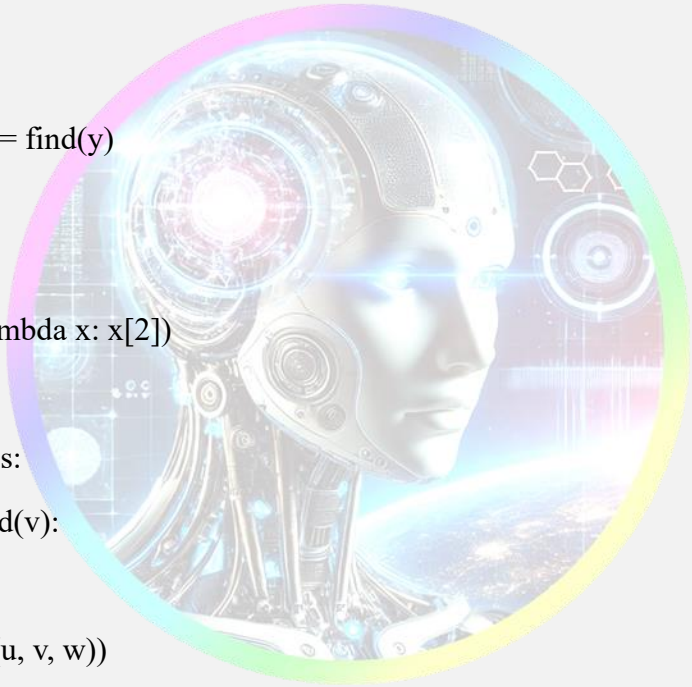
Kruskal's Algorithm builds MST by sorting edges and adding the **smallest edge** that doesn't form a cycle using **Union-Find**.

Steps:

1. Sort all edges.
2. Initialize each node as its own set.
3. Add edge if nodes belong to different sets.
4. Merge sets.

Python Code:

```
def kruskal(nodes, edges):
    parent = {n: n for n in nodes}

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    def union(x, y):
        parent[find(x)] = find(y)

    mst = []
    edges.sort(key=lambda x: x[2])

    for u, v, w in edges:
        if find(u) != find(v):
            union(u, v)
            mst.append((u, v, w))
    return mst
```



11. What is the time complexity of Kruskal's algorithm? Justify your answer.

- **Sorting edges**: O(E log E)
- **Union-Find operations** (with path compression and union by rank): ~O(α(V)) per operation, where α is the inverse Ackermann function (very small).

Total Time Complexity:
O(E log E) or **O(E log V)** (since E ≥ V - 1 in a connected graph)

---

12. Compare and contrast Prim's and Kruskal's algorithms. When would you prefer one over the other?

Feature	Prim's	Kruskal's
Approach	Grows MST from one node	Adds smallest edge globally
Data structure	Min-heap (priority queue)	Union-Find
Time complexity	O(E log V)	O(E log E)
Best for	Dense graphs	Sparse graphs
Cycle check	Not needed	Needed via Union-Find

Preference:

- Use **Prim's** for dense graphs with many edges.
- Use **Kruskal's** for sparse graphs with fewer edges.

---

13. Explain the concept of a spanning tree. How is it different from a minimum spanning tree?

- **Spanning Tree**: A subgraph that includes all vertices and is a tree (no cycles, connected).
- **Minimum Spanning Tree (MST)**: A spanning tree with the **least total weight**.

Difference:

- All MSTs are spanning trees.
- Not all spanning trees are minimal in weight.

---

14. What is the difference between a graph and a tree? Provide examples.

Feature	Graph	Tree
Cycles	May contain	No cycles
Connectivity	May be disconnected	Always connected
Edges	Varies	Always V-1
Types	Directed/Undirected	Rooted, binary, etc.

Example:

- **Graph**: Social network.
- **Tree**: File system hierarchy.

---

15. Explain how to detect a cycle in a graph using DFS. Provide Python code.

For Undirected Graph:
Track parent during DFS to avoid false cycles.

```
def has_cycle(graph, node, visited, parent):
    visited.add(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            if has_cycle(graph, neighbor, visited, node):
                return True
        elif neighbor != parent:
            return True
    return False
```

---

16. What is a bipartite graph? How do you check if a graph is bipartite?

A **bipartite graph** is one whose vertices can be divided into **two disjoint sets** such that no two vertices within the same set are adjacent.

Check using BFS with 2-coloring:

```
from collections import deque


def is_bipartite(graph):
    color = {}
    for node in graph:
        if node not in color:
            queue = deque([node])
            color[node] = 0
```

```
        while queue:
            curr = queue.popleft()
            for neighbor in graph[curr]:
                if neighbor not in color:
                    color[neighbor] = 1 - color[curr]
                    queue.append(neighbor)
                elif color[neighbor] == color[curr]:
                    return False
    return True
```

---

17. Explain the concept of a weighted graph. How is it different from an unweighted graph?

- A **weighted graph** has edges with weights (e.g., distance, cost).
- An **unweighted graph** assumes all edges have equal weight (typically 1).

Use case:

- Weighted: Road networks (distances)
- Unweighted: Friend suggestions on social media (just connections)

---

18. What is the difference between a directed and an undirected graph? Provide examples.

Graph Type	Edge Direction	Example
Directed	A → B only	Web links (A links to B)
Undirected	A ↔ B	Social connections (friends)

In a **directed graph**, edges have direction. In **undirected graphs**, edges are bidirectional.

---

19. Explain the concept of a connected graph. How do you check if a graph is connected?

- A **connected graph** is one in which there is a path between every pair of vertices.

Check using DFS or BFS:

- Start from a node, mark all reachable nodes.
- If all nodes are visited, the graph is connected.

```
def is_connected(graph, start):
    visited = set()

    def dfs(node):
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                dfs(neighbor)

    dfs(start)
    return len(visited) == len(graph)
```

---

20. What is the difference between a graph and a network? Provide examples.

- A **graph** is a general data structure representing nodes and connections.
- A **network** is a **real-world application** of a graph, often with additional semantics (e.g., flow, bandwidth).

Term	Usage Example
Graph	Abstract data model
Network	Computer networks, transport systems

Example:

- Graph: A structure with vertices and edges.
- Network: The internet (routers and links form a graph-like structure with real-world meaning).

Data Structure : Assignment 5B

1. Explain the concept of linear search. Provide an example and Python code.

Concept: Linear search checks each element in the list until it finds the target element or reaches the end.

Python Example:

```
def linear_search(arr, x):
    for i in range(len(arr)):
        if arr[i] == x:
            return i
    return -1

# Example usage
arr = [10, 20, 30, 40, 50]
print(linear_search(arr, 30))  # Output: 2
```

---

2. Explain the concept of binary search. Provide an example and Python code.

Concept: Binary search works on sorted arrays and repeatedly divides the search interval in half.

Python Example:

```
def binary_search(arr, x):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == x:
            return mid
        elif arr[mid] < x:
            low = mid + 1
        else:
```

```
            high = mid - 1
    return -1


# Example usage
arr = [5, 10, 15, 20, 25]
print(binary_search(arr, 15))  # Output: 2
```

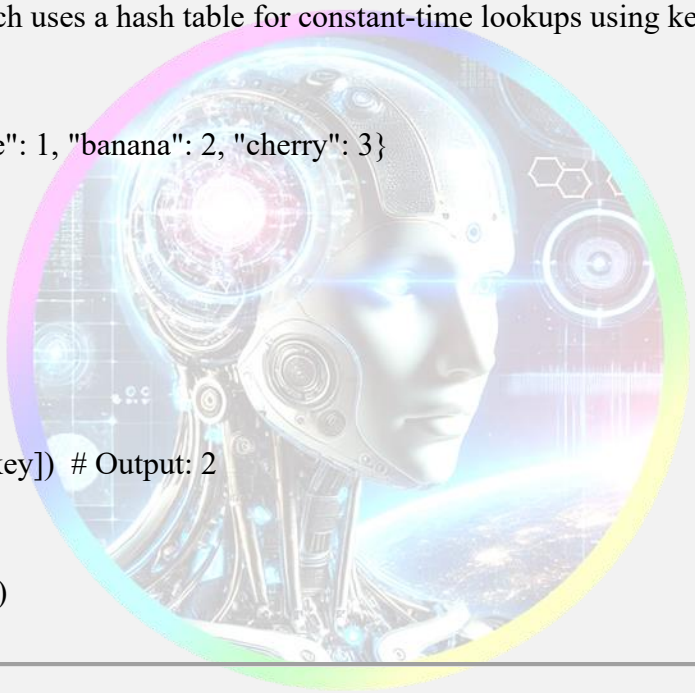
---

3. Explain the concept of hash search. Provide an example and Python code.

Concept: Hash search uses a hash table for constant-time lookups using keys.

Python Example:

```
hash_table = {"apple": 1, "banana": 2, "cherry": 3}


# Example usage
key = "banana"
if key in hash_table:
    print(hash_table[key])  # Output: 2
else:
    print("Not found")
```



---

4. Explain the concept of insertion sort. Provide an example and Python code.

Concept: Builds a sorted list one element at a time by inserting elements into their correct position.

Python Example:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
```

```
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

print(insertion_sort([4, 3, 2, 1]))  # Output: [1, 2, 3, 4]
```

---

5. Explain the concept of selection sort. Provide an example and Python code.

Concept: Repeatedly selects the minimum element from the unsorted part and moves it to the sorted part.

Python Example:

```
def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i+1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

print(selection_sort([64, 25, 12, 22, 11]))  # Output: [11, 12, 22, 25, 64]
```

---

6. Explain the concept of bubble sort. Provide an example and Python code.

Concept: Repeatedly swaps adjacent elements if they are in the wrong order.

Python Example:

```
def bubble_sort(arr):
```

```
    n = len(arr)
    for i in range(n):
        for j in range(n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

print(bubble_sort([5, 1, 4, 2, 8]))  # Output: [1, 2, 4, 5, 8]
```

---

7. Explain the concept of quick sort. Provide an example and Python code.

Concept: Divides array into two smaller arrays using a pivot and recursively sorts them.

Python Example:

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    less = [x for x in arr[1:] if x <= pivot]
    greater = [x for x in arr[1:] if x > pivot]
    return quick_sort(less) + [pivot] + quick_sort(greater)

print(quick_sort([10, 7, 8, 9, 1, 5]))  # Output: [1, 5, 7, 8, 9, 10]
```

---

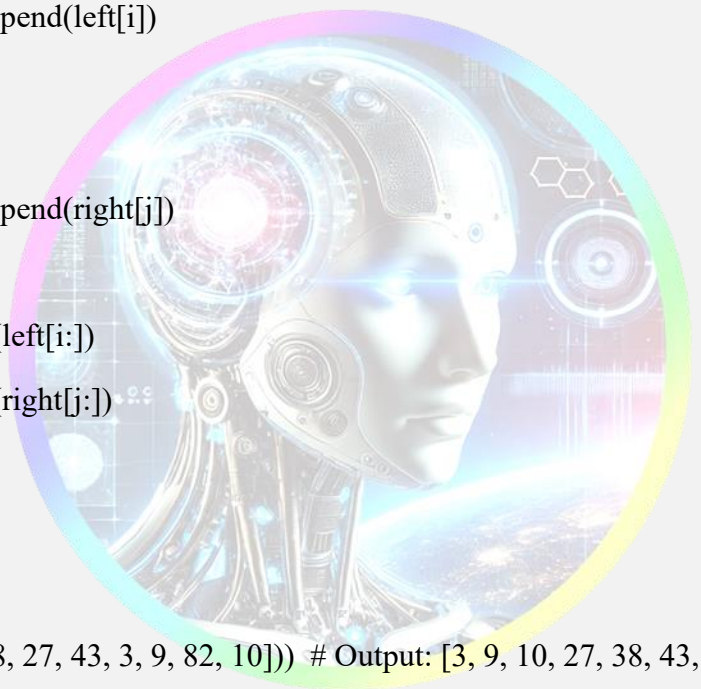
8. Explain the concept of merge sort. Provide an example and Python code.

Concept: Recursively divides the array and merges the sorted subarrays.

Python Example:

```
def merge_sort(arr):
    if len(arr) > 1:
```

```
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    merged = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
  return arr

print(merge_sort([38, 27, 43, 3, 9, 82, 10]))  # Output: [3, 9, 10, 27, 38, 43, 82]
```



---

9. Explain the concept of heap sort. Provide an example and Python code.

Concept: Converts array to a heap and repeatedly removes the max element to sort.

Python Example:

```
import heapq

def heap_sort(arr):
    heapq.heapify(arr)
```

```
    sorted_list = [heapq.heappop(arr) for _ in range(len(arr))]
    return sorted_list

print(heap_sort([12, 11, 13, 5, 6, 7]))  # Output: [5, 6, 7, 11, 12, 13]
```

---

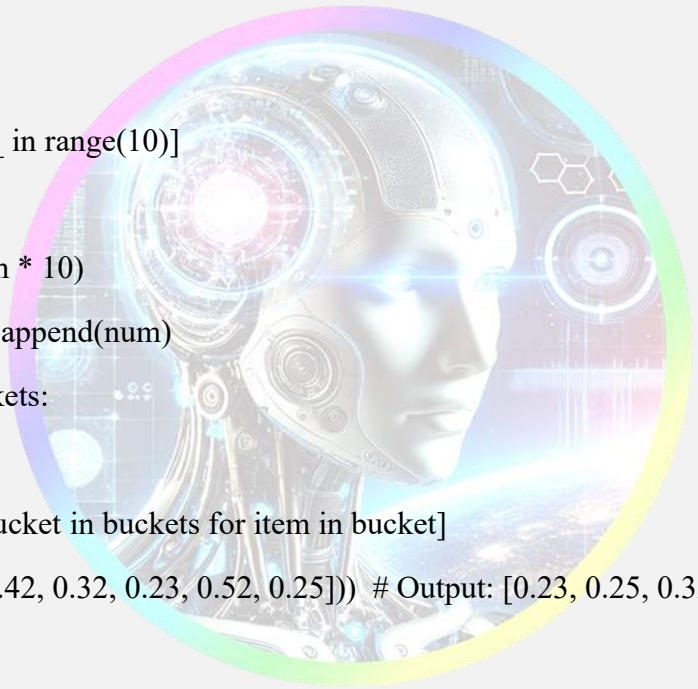
10. Explain the concept of bucket sort. Provide an example and Python code.

Concept: Divides elements into buckets, sorts each bucket, and merges them.

Python Example:

Python code;

```
def bucket_sort(arr):
    buckets = [[] for _ in range(10)]
    for num in arr:
        index = int(num * 10)
        buckets[index].append(num)
    for bucket in buckets:
        bucket.sort()
    return [item for bucket in buckets for item in bucket]
print(bucket_sort([0.42, 0.32, 0.23, 0.52, 0.25]))  # Output: [0.23, 0.25, 0.32, 0.42, 0.52]
```



11. Compare and contrast linear search and binary search. When would you prefer one over the other?

Feature	Linear Search	Binary Search
Data Requirement	Works on unsorted data	Requires sorted data
Time Complexity	O(n)	O(log n)
Space Complexity	O(1)	O(1)
Use Case	Small or unsorted data	Large, sorted datasets
Simplicity	Very simple to implement	Slightly complex due to mid calculations

Prefer Linear Search when the dataset is small or unsorted.
Prefer Binary Search when the dataset is large and sorted.

---

12. Compare and contrast insertion sort and selection sort. When would you prefer one over the other?

Feature	Insertion Sort	Selection Sort
Time Complexity	$O(n^2)$	$O(n^2)$
Best Case	O(n) (already sorted)	$O(n^2)$ (even if sorted)
Stability	Stable	Not stable
Memory	In-place	In-place
Preference	When data is nearly sorted	When memory write operations are costly

Prefer Insertion Sort when data is nearly sorted.
Prefer Selection Sort when minimizing the number of swaps is critical.

---

13. Compare and contrast bubble sort and quick sort. When would you prefer one over the other?

Feature	Bubble Sort	Quick Sort
Time Complexity	$O(n^2)$	Average: O(n log n), Worst: $O(n^2)$
Stability	Stable	Not stable
Space Complexity	O(1)	O(log n) (recursive calls)
Simplicity	Very easy to implement	More complex

Prefer Bubble Sort for educational purposes or very small datasets.
Prefer Quick Sort for large datasets needing performance.

---

14. Compare and contrast merge sort and heap sort. When would you prefer one over the other?

Feature	Merge Sort	Heap Sort

Time Complexity	O(n log n)	O(n log n)
Space Complexity	O(n)	O(1) (in-place)
Stability	Stable	Not stable
Parallelizable	Yes	No

Prefer Merge Sort for linked lists or when stability is needed.
Prefer Heap Sort for memory-limited environments.

---

15. Compare and contrast quick sort and merge sort. When would you prefer one over the other?

Feature	Quick Sort	Merge Sort
Time Complexity	Avg: O(n log n), Worst: $O(n^2)$	O(n log n)
Space Complexity	O(log n)	O(n)
Stability	Not stable	Stable
Cache Performance	Excellent	Poorer due to recursion and merging

Prefer Quick Sort for in-place sorting and speed.
Prefer Merge Sort when stable sort is needed or working with linked lists.

---

16. Compare and contrast heap sort and bucket sort. When would you prefer one over the other?

Feature	Heap Sort	Bucket Sort
Time Complexity	O(n log n)	Avg: O(n + k), Worst: $O(n^2)$
Space Complexity	O(1)	O(n + k)
Data Type	Any data	Works best with uniform distribution
Stability	Not stable	Can be made stable

Prefer Heap Sort for general-purpose sorting.
Prefer Bucket Sort for floating-point or uniformly distributed data.

---

17. Explain the concept of stable and unstable sorting algorithms. Provide examples.

Stable Sort: Maintains the relative order of equal elements.

- **Examples**: Merge Sort, Insertion Sort, Bubble Sort.

Unstable Sort: May not preserve the relative order of equal elements.

- **Examples**: Quick Sort, Selection Sort, Heap Sort.

Use stable sort when order matters (e.g., sorting records by age but keeping same-name order).

---

18. Explain the concept of in-place and out-of-place sorting algorithms. Provide examples.

- **In-place Sort**: Uses constant space, modifies the original array.
    - **Examples**: Quick Sort, Bubble Sort, Insertion Sort.
- **Out-of-place Sort**: Uses extra space for sorting.
    - **Examples**: Merge Sort, Bucket Sort.

Use in-place for memory efficiency.
Use out-of-place for avoiding side effects or complex merging.

---

19. Explain the concept of internal and external sorting algorithms. Provide examples.

- **Internal Sort**: All data fits in RAM.
    - **Examples**: Quick Sort, Merge Sort (for small data).
- **External Sort**: Data is too large for memory, uses disk.
    - **Examples**: External Merge Sort, Replacement Selection.

External sorting is used in databases, big data, and disk-heavy operations.

---

20. Explain the concept of comparison-based and non-comparison-based sorting algorithms. Provide examples.

- **Comparison-Based**: Elements are compared using operators (<, >, etc.)
    - **Examples**: Merge Sort, Quick Sort, Heap Sort.
    - **Lower bound**: O(n log n)
- **Non-Comparison-Based**: Use keys or other characteristics (no direct comparison).
    - **Examples**: Counting Sort, Bucket Sort, Radix Sort.

- **Can achieve**: O(n) in best case.

Use **non-comparison** sorts for integers or small key ranges.
Use **comparison** sorts for general-purpose and large datasets.